

Irish Tax Simulator - Educational Guide

Overview

This project is an interactive tax simulator that demonstrates the Irish tax system using real income distribution data from 2023. It's designed to help students learn web development fundamentals while exploring economics and tax policy.

Project Files

- `index.html` - The structure and content of the webpage
 - `styles.css` - The visual styling and layout
 - `script.js` - The logic, calculations, and interactivity
 - `README.md` - This guide (documentation)
-

What is HTML?

HTML (HyperText Markup Language) is the skeleton of every webpage. It defines the structure and content.

Key Concepts:

- **Elements:** Building blocks like `<div>`, `<p>`, `<button>`
- **Attributes:** Extra information like `id="myButton"` or `class="slider-group"`
- **Nesting:** Elements can contain other elements
- **Semantic HTML:** Using meaningful tags like `<header>`, `<section>`, `<nav>`

In This Project:

```
```html
```

```
... 
```

```
```
```

The `<canvas>` elements are where Chart.js draws the graphs!

What is CSS?

CSS (Cascading Style Sheets) controls how HTML elements look.

Key Concepts:

- **Selectors:** Target elements to style (`.class`, `#id`, `element`)
- **Properties:** What to change (`color`, `font-size`, `margin`)
- **Values:** What to change it to (`red`, `16px`, `1rem`)
- **Box Model:** Every element has content, padding, border, and margin

In This Project:

```
.slider-group {                /* Select all elements with
class="slider-group" */
    margin-bottom: 1.2rem;      /* Space below each slider */
}

button:hover {                 /* When mouse hovers over button */
    background: #41649c;        /* Change background color */
}
```

CSS Units:

- `px` - Pixels (fixed size)
- `rem` - Relative to root font size (responsive)
- `%` - Percentage of parent element
- `vh` / `vw` - Viewport height/width

What is JavaScript?

JavaScript makes webpages interactive and dynamic.

Key Concepts:

- **Variables:** Store data (`const`, `let`, `var`)
- **Functions:** Reusable blocks of code
- **Arrays:** Lists of items `[1, 2, 3]`
- **Objects:** Collections of properties `{name: "John", age: 30}`

- **Events:** Respond to user actions (clicks, typing, etc.)
- **DOM Manipulation:** Change HTML/CSS with JavaScript

In This Project:

```
// Variable storing data
const USC_EXEMPTION = 13000.0;

// Function that does calculations
function calculateIncomeTax(income, thresholds, rates, credit) {
    // ... calculation logic ...
    return totalTax;
}

// Event listener – runs when slider moves
document.getElementById('band1').addEventListener('input',
recalculateAll);
```

How to Run This Project

Option 1: Simple (Double-click)

1. Make sure all three files (`index.html`, `styles.css`, `script.js`) are in the same folder
2. Double-click `index.html`
3. It should open in your default web browser

Option 2: Using VS Code Live Server

1. Install the “Live Server” extension in VS Code
2. Right-click `index.html`
3. Select “Open with Live Server”
4. Changes will update automatically!

Option 3: Python Simple Server

```
# In the project folder, run:
python3 -m http.server 8000

# Then open: http://localhost:8000
```

Understanding the Tax Simulator

The Irish Tax System

Ireland has three main components of income tax:

1. **Income Tax** (Progressive Brackets)
 - Standard rate: 20% up to €44,000
 - Higher rate: 40% above €44,000
 - Tax credit: €2,000 subtracted from final tax
2. **USC (Universal Social Charge)**
 - Exempt if income < €13,000
 - Progressive rates: 0.5%, 2%, 3%, 8%
3. **PRSI (Pay Related Social Insurance)**
 - 4.1% of income
 - Credit for low earners (€352-€424/week)

Key Terms

- **Effective Tax Rate:** Total tax paid ÷ gross income
- **Progressive Tax:** Higher earners pay higher percentage
- **Tax Credit:** Fixed amount subtracted from tax bill
- **Tax Bracket:** Income ranges with different rates

How the Simulator Works

1. **Real Data:** Uses actual 2023 income distribution (3.9M+ earners)
2. **Average Income:** Each band has average income per person
3. **Tax Calculation:** Applies tax rules to each income band
4. **Aggregation:** Multiplies by number of earners
5. **Visualization:** Shows results in charts

Exercises

The code includes commented-out exercises at three skill levels:

Beginner Exercises (These are the ones we care about)

Exercise 1: Change Colors

File: `styles.css`

Task: Uncomment the header background color and change it

```
css header { background-color: #4C72B0; /* Try different colors!
*/ }
```

Exercise 2: Add Hover Effect

File: `styles.css`

Task: Uncomment the button transform on hover

```
css button:hover { transform: translateY(-2px); /* Button lifts
up */ }
```

Exercise 3: Display Average Tax

File: `script.js`

Task: Uncomment the `calculateAverageTaxPerPerson()` function and display the result in the summary

Intermediate Exercises

Exercise 4: Make it Responsive

File: `styles.css`

Task: Uncomment the media query and adjust styles for mobile screens

Exercise 5: Calculate Median Tax Rate

File: `script.js`

Task: Complete the `calculateMedianEffectiveTaxRate()` function

Exercise 6: Add a Comparison Feature

Files: `index.html`, `styles.css`, `script.js`

Task:

1. Uncomment the comparison section in HTML

2. Uncomment the comparison styles in CSS
3. Complete the `saveCurrentScenario()` and `compareToSavedScenario()` functions in JavaScript

Advanced Exercises

Exercise 7: Revenue by Income Band

File: `script.js`

Task: Create a new chart showing which income groups contribute most tax revenue

- Use the `calculateRevenueByBand()` function
- Create a bar chart with Chart.js
- Add it to the HTML

Exercise 8: Progressivity Index

File: `script.js`

Task: Calculate and display how progressive the tax system is

- Complete the `calculateProgressivityIndex()` function
- Display the index in the UI
- Explain what it means (higher = more progressive)

Exercise 9: Dark Mode

Files: `styles.css`, `script.js`

Task: Implement a complete dark mode

1. Uncomment dark mode styles in CSS
2. Complete the `toggleDarkMode()` function
3. Add a toggle button to HTML
4. Update Chart.js colors for dark mode
5. Save user preference in localStorage

Code Deep Dive

How Sliders Work

1. HTML defines the slider:

```
html <input type="range" id="band1" min="10000" max="100000"
```

```
value="44000">
```

2. **JavaScript** listens for changes:

```
javascript  
document.getElementById('band1').addEventListener('input',  
recalculateAll);
```

3. **Function** gets the value and recalculates:

```
javascript const band1 =  
parseFloat(document.getElementById('band1').value);
```

How Charts Update

1. **Initial creation** with Chart.js:

```
javascript const revenueChart = new Chart(context, { type: 'bar',  
data: {...}, options: {...} });
```

2. **Update data** when sliders change:

```
javascript revenueChart.data.datasets[0].data[1] = newRevenue;  
revenueChart.update(); // Redraw the chart
```

Tax Calculation Flow

```
User Moves Slider  
  ↓  
recalculateAll() runs  
  ↓  
Get all slider values  
  ↓  
Calculate taxes:  
- calculateIncomeTax()  
- calculateUSC()  
- calculatePRSI()  
  ↓  
Calculate total revenue  
  ↓  
Update charts  
  ↓  
Update summary text
```

Customization Ideas

Easy Changes:

- Modify color scheme (search for hex colors like `#4C72B0`)
- Add more text explanations
- Change default slider values
- Adjust chart heights

Medium Changes:

- Add new sliders (e.g., for USC rate)
- Create additional charts
- Add tax calculator for individual income
- Export data to CSV

Advanced Changes:

- Add multiple country tax systems
 - Implement optimization algorithms
 - Create scenario presets (e.g., “Nordic Model”, “Flat Tax”)
 - Add animation when values change
 - Build a backend to save scenarios
-

Further Learning Resources

HTML & CSS:

- [MDN Web Docs](#) - Comprehensive reference
- [CSS-Tricks](#) - Practical CSS tutorials
- [Flexbox Froggy](#) - Learn flexbox through games

JavaScript:

- [JavaScript.info](#) - Modern JavaScript tutorial
- [Eloquent JavaScript](#) - Free online book
- [FreeCodeCamp](#) - Interactive courses

Chart.js:

- [Chart.js Documentation](#) - Official docs
- [Chart.js Examples](#) - Code samples

Git & Version Control:

- [Git Documentation](#) - Learn version control
 - [GitHub Guides](#) - Collaborative development
-

Debugging Tips

Common Issues:

Charts not showing?

- Check browser console for errors (F12)
- Ensure Chart.js CDN is loading
- Verify canvas elements have correct IDs

Calculations seem wrong?

- Add `console.log()` statements
- Check that values are numbers not strings
- Use browser debugger to step through code

Styling not applying?

- Check CSS file is linked correctly
- Inspect element (right-click → Inspect)
- Look for typos in class/id names
- Check CSS specificity rules

JavaScript not running?

- Check browser console for errors
- Ensure script.js is linked with `defer`
- Check for syntax errors (missing brackets, semicolons)

Browser Developer Tools:

- **F12** - Open developer tools
 - **Console tab** - See errors and logs
 - **Elements tab** - Inspect HTML/CSS
 - **Sources tab** - Debug JavaScript
 - **Network tab** - Check if files load
-

💡 Project Ideas for Extension

1. **Compare Countries:** Add US, UK, German tax systems
 2. **Historical Analysis:** Show how Irish tax changed over time
 3. **Impact Calculator:** “How would this affect someone earning €X?”
 4. **Inequality Metrics:** Calculate Gini coefficient
 5. **Mobile App:** Convert to React Native
 6. **Data Visualization:** Add more charts (pie, scatter, etc.)
 7. **API Integration:** Pull live tax rates
 8. **User Accounts:** Save and share scenarios
-

License & Data

- **Code:** Educational use (modify freely)
 - **Data:** Irish Revenue 2023 income distribution (public data)
 - **Chart.js:** Open source (MIT License)
-

FAQ

Q: Do I need to install anything?

A: Just a web browser and text editor! Chart.js loads from a CDN.

Q: Can I change the income data?

A: Yes! Modify the `incomeBands` array in `script.js`. Just be sure to maintain the same structure—though also see what happens if you don't!

Q: Why are there three files?

A: Separation of concerns - structure (HTML), presentation (CSS), and behavior (JS) are separate. This is a best practice in web development.

Q: What if I break something?

A: Make a copy of the original files first! Or use Git to track changes.

Q: How do I add more tax brackets?

A: The system already supports up to 4 brackets (band1, band2, band3). You can add more by following the existing pattern in the HTML and JavaScript.

Q: Can I use this for my own country?

A: Absolutely! Replace the income data and adjust the tax calculation functions.

Support

If you encounter issues or have questions:

1. Check the comments in the code
 2. Review this README carefully
 3. Use browser developer tools to debug
 4. Try to avoid using a search engine like google or ChatGPT, instead, look at one of the tutorials we have used
 5. Ask your instructor or peers
-

Happy Coding!

Remember: The best way to learn is by doing. Don't be afraid to experiment, break things, and try again. Every error is a learning opportunity!